1) Insert 15 at beginning

① int arr[5] = {20, 30, 40, 50}

int n=4;

OR ~~arr~~ for (int i=n ; i>=0 ; i--)
{ arr[i] = arr[i-1] ; }

itr 1	i=4	arr[4] = arr[3]	20 30 40 50 **50**
itr 2	i=3	arr[3] = arr[2]	20 30 40 **40** 50
itr 3	i=2	arr[2] = arr[1]	20 30 **30** 40 50
itr 4	i=1	arr[1] = arr[0]	20 **20** 30 40 50
itr 5	~~i=0~~	arr[0] = arr[-]	15 20 30 40 50

Output, 15, 20, 30, 40, 50

2) Insert 60 at end

int arr[6] = {10, 20, 30, 40, 50}
int n=5

arr[n] = 60;
n++

arr[5] = 60 n=6

for ~~after~~ display
for(int i=0; i<n ; i++)
{ cout << arr[i] << " " ; }

itr 1	i = 0	10
2	i = 1	20
3	i = 2	30
4	i = 3	40
5	i = 4	50
6	i = 5	60
7	i = 6	Condition i < n is false

Output => 10 20 30 40 50 60

3) Index 25 at index 2

```
int arr[5] = {10, 20, 30, 40}
int n = 4
index = 2 , value = 25
```

for shift elements to right

```
for (int i = n ; i > index ; i--) {
{ arr[i] = arr[i-1] ;
```

itr 1	i = 4	arr[4] = arr[3]	10 20 30 40 40
itr 2	i = 3	arr[3] = arr[2]	10 20 30 30 40
itr 3	i = 2	Condition i > index // index = 2 is false.	10 20 30 30 40

arr[index] = value

arr[2] = 25

n++

n+1 => 4+1 => n = 5

CamScanner

for displaying

```
for (int i=0 ; i<n ; i++)
{   cout << arr[i] << " " ; }
```

itr 1	i = 0	arr[0]	10
itr 2	i = 1	arr[1]	20
itr 3	i = 2	arr[2]	25
itr 4	i = 3	arr[3]	30
itr 5	i = 4	arr[4]	40
itr 6	i = 5	Condition become false	

Output => 10 20 25 30 40

4) Delete the first element

```
int arr[5] = {10, 20, 30, 40, 50}
int n = 5
```

for shifting elements to LEFT

```
for (int i=0 ; i<n-1 ; i++)
{   arr[i] = arr[i+1] ; }
```

itr 1	i = 0	arr[0] = arr[1]	20 20 30 40 50
2	i = 1	arr[1] = arr[2]	20 30 30 40 50
3	i = 2	arr[2] = arr[3]	20 30 40 40 50
4	i = 3	arr[3] = arr[4]	20 30 40 50 50
5	i = 4	Condition i<4 is false	

CamScanner

n-- ... n = 5-1 ; n = 4

for displaying

i = 0	arr[0]	20
i = 1	arr[1]	30
i = 2	arr[2]	40
i = 3	arr[3]	50
i = 4	Condition false.	

Output 20 30 40 50

02

5) Delete the last Element

```
int arr[5] = {10, 20, 30, 40, 50}
int n = 5;

n--;        n = 5-1 ; n = 4
```

for display array.

```
for(i=0; i<n; i++)
{ cout << arr[i] ; }
```

itr 1	i = 0	arr[0]	10
2	i = 1	arr[1]	20
3	i = 2	arr[2]	30
4	i = 3	arr[3]	40
5	i = 4	Condition False	

Output : 10 20 30 40

CamScanner

6) Delete the element at index 2.

arr[5] = { 10, 20, 30, 40, 50 }

n = 5

index = 2

Shift elements to Left.

```
for (int i=index ; i < n-1 ; i++)
  { arr[i] = arr[i+1] ; }
```

itr 1	i = 2	arr[2] = arr[3]	10	20	40	40	50
2	i = 3	arr[3] = arr[4]	10	20	40	50	50
3	i = 4	Condition i < 4 false					

n--; n = 5-1, n = 4

Display Array

itr 1	i = 0	arr[0]	10
2	i = 1	arr[1]	20
3	i = 2	arr[2]	40
4	i = 3	arr[3]	50
5	i = 4	Condition	false

Output : 10 20 40 50.

CamScanner

7) Find 40, using Linear Search

Checks each element one-by-one from left to right until it finds.

arr[5] = {10, 20, 30, 40, 50}

n = 5 , key = 40

found = -1

for (int i = 0; i < n; i++)

if (arr[i] == key)

itr 1	i = 0	arr[0]	10 == 40	Continue
2	i = 1	arr[1]	20 == 40	"
3	i = 2	arr[2]	30 == 40	"
4	i = 3	arr[3]	40 == 40	found
5	i = 4	~~arr~~	break	

if (found != -1)

Condition found != -1 is true because found = 3

Output :

Value found at index 3

index 3 is fourth position because indexing starts from zero.

CamScanner

8) Find 40 using Binary Search.

Binary Search checks the middle element and eliminates half of the remaining search area each time.

arr[5] = {10, 20, 30, 40, 50}

n = 5

key = 40

low = 0

high = n-1 ; & found = -1 ;

itr1 mid = $\frac{low + high}{2}$ => $\frac{0+4}{2}$ = 2

arr[2] = 30

check condition (arr[mid] == key)
(arr[mid] < key)

30 == 40 NO

30 < 40 Yes

low = mid + 1 = 2 + 1 , 3

low = 3

search area is now indexes 3 to 4.

itr 2 $mid = \frac{3+4}{2} = \frac{7}{2}, 3.5 \approx 3$

arr [mid] == key

arr [3] == 40

40 == 40

found = mid

found = 3

Output → Value found at index 3